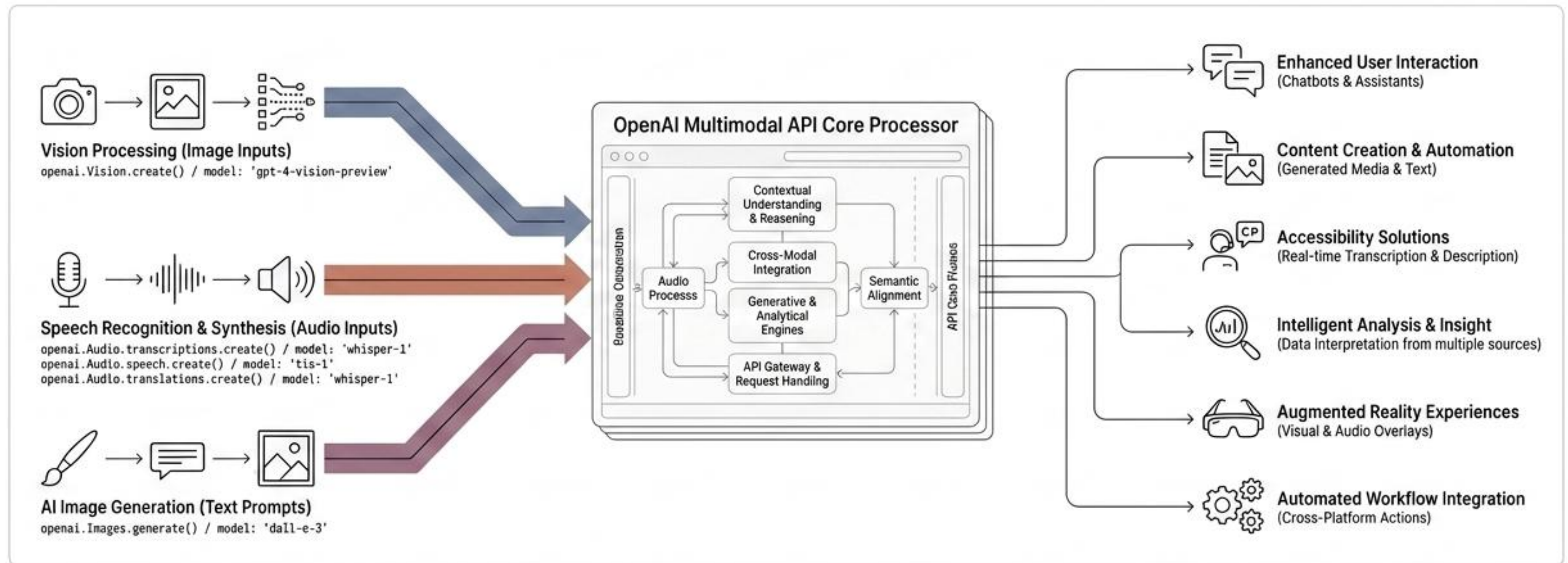


Developing Multimodal AI Applications with OpenAI APIs

Vision Processing, Speech Synthesis, Speech Recognition, and AI Image Generation



Lesson Objectives



Define the architecture of a multimodal AI application



Process image inputs using OpenAI Vision capabilities



Convert AI textual responses into spoken audio



Transcribe user speech into machine-readable text



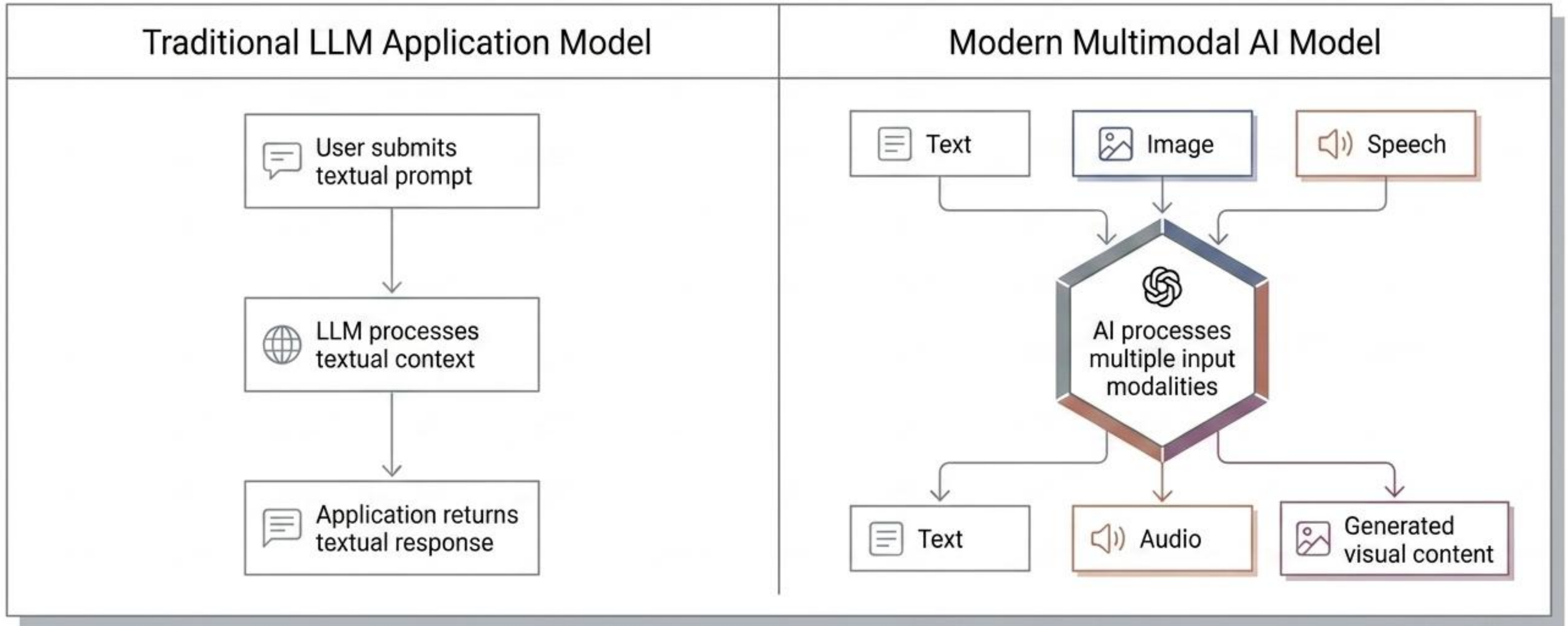
Generate images dynamically from textual prompts



Combine multiple OpenAI APIs into one integrated software solution

Moving from text-only chatbots to AI applications that can see, hear, speak, and create.

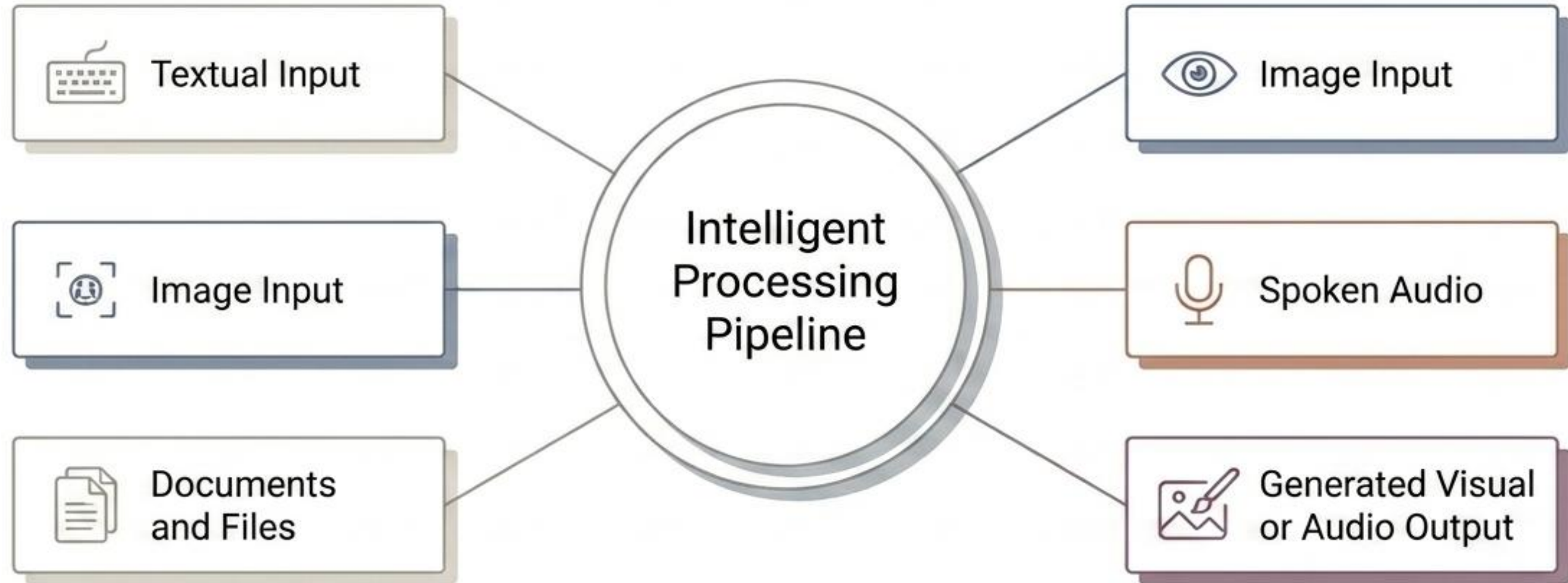
From Text Applications to Multimodal AI Systems



Applications become significantly more interactive, intelligent, and human-centered.

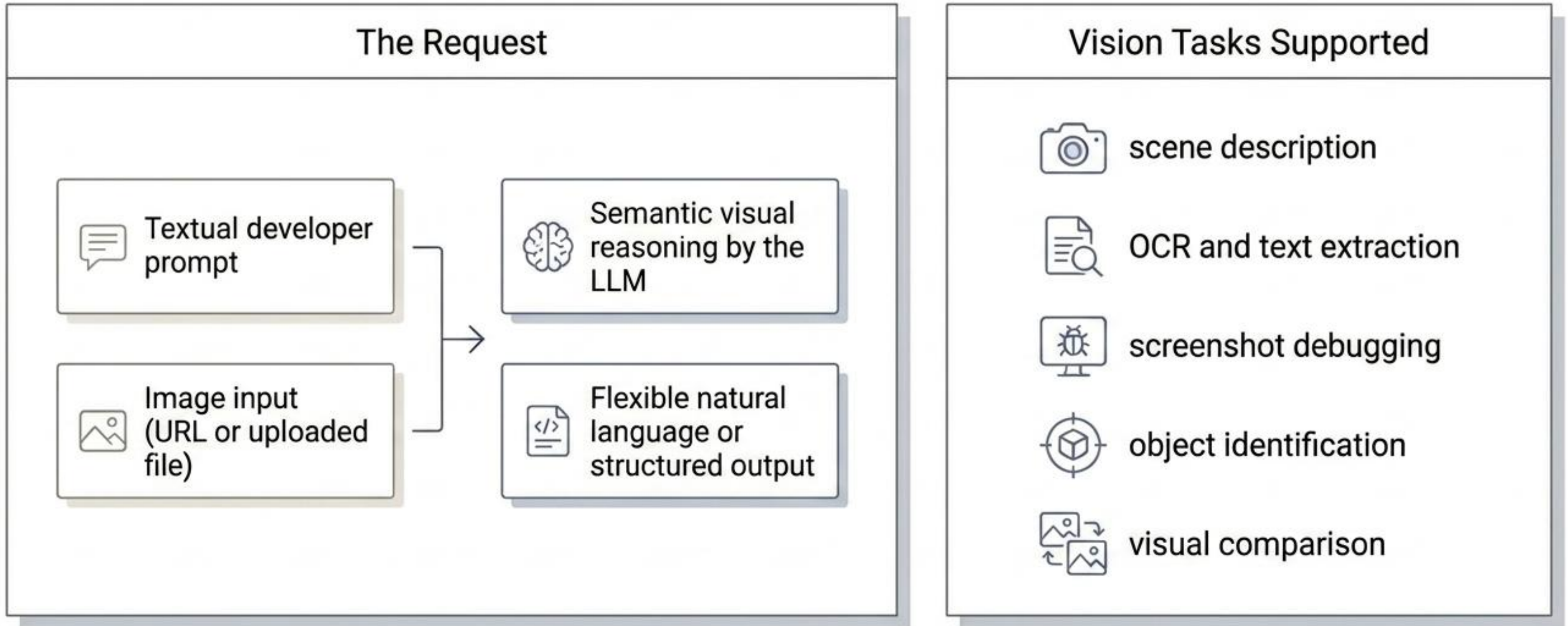
What Makes an Application Multimodal?

A **modality** is a distinct form of digital information processed by the system.



A multimodal AI application combines two or more modalities within one intelligent processing pipeline.

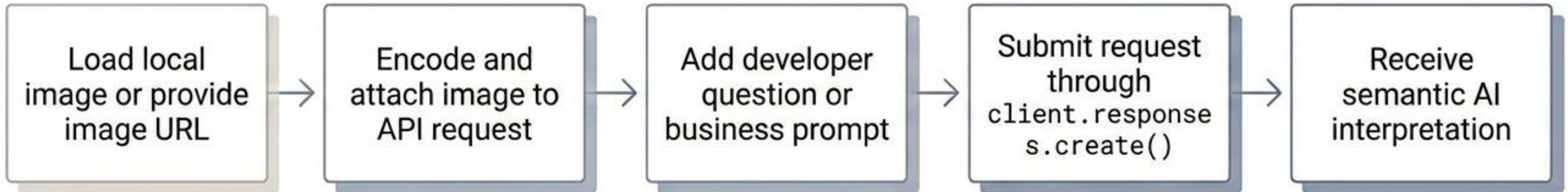
OpenAI Vision Processing with Responses API



One API call enables intelligent understanding of visual content without training a custom vision model.

Vision API Development Workflow

Python Processing Pipeline



Example Developer Prompts

- > Describe the content of this image
- > Extract all visible text
- > Detect the main issue shown in this screenshot
- > Compare this image to expected product condition

Vision API Python Implementation

vision_api.py

```
1  from openai import OpenAI
2  client = OpenAI()
3  response = client.responses.create(
4      model="gpt-5",
5      input=[
6          {
7              "role": "user",
8              "content": [
9                  {
10                     "type": "input_text",
11                     "text": "What do you see in the image?",
12                 },
13                 {
14                     "type": "input_image",
15                     "image_url": "https://api.nga.gov/iiif/a2e6da57-3cd1-4235-b20e-95dcaefed6c8/full/1800,800/0/default.jpg"
16                 }
17             ]
18         }
19     ]
20 )
21 print(response.output_text)
```

Real-World Vision Application Opportunities

Using the same Vision API, developers can build:



Smart screenshot error analyzers



Receipt and invoice readers



Product quality inspection tools



Classroom or meeting
image summarizers



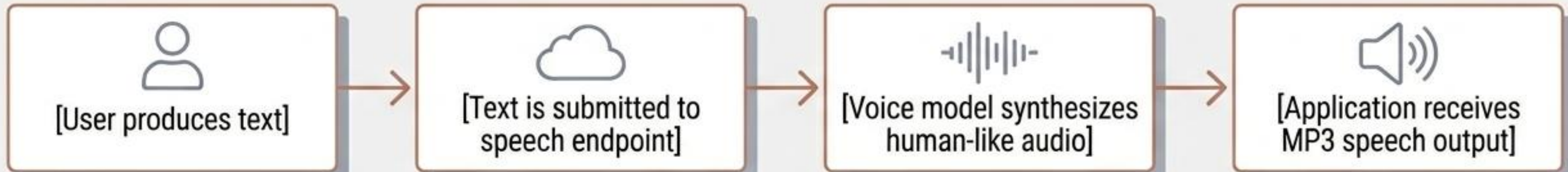
Intelligent document
scanning assistants

The image remains the same, only the prompt changes, allowing one multimodal backend to support multiple software products.

OpenAI Text-to-Speech API

Text-to-Speech enables the application to transform any generated textual response into natural spoken audio.

Core Processing Flow



Common Development Uses

Talking virtual assistants

Accessibility support systems

Spoken notifications

Educational reading applications

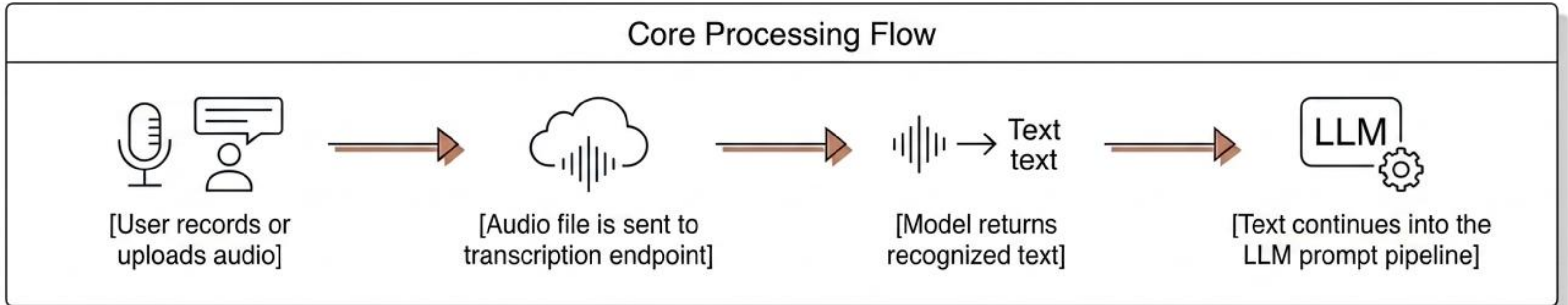
AI responses are no longer limited to screen text, they become audible interaction.

Text-to-Speech Python Implementation

Project	tts_api.py ×
src	
tts_api.py	
speech.mp3	
	<pre>1 from openai import OpenAI 2 3 client = OpenAI() 3 with client.audio.speech.with_streaming_response.create(4 model="gpt-4o-mini-tts", 5 voice="coral", 6 input="Hello students, welcome to our amazing course", 7 instructions="Speak in a cheerful and positive tone." 8) as response: 9 response.stream_to_file("speech.mp3") response.stream_to_file("speech.mp3")</pre>

OpenAI Speech-to-Text API

Speech-to-Text enables the application to convert human voice into machine-readable text for downstream LLM processing.



Common Development Uses

Voice command interfaces

Meeting transcription systems

Hands-free chatbot interaction

Audio note analyzers

Applications can now receive natural spoken user input.

Speech-to-Text Python Implementation

stt_api.py x

```
1 from openai import OpenAI
2 client = OpenAI()
3 audio_file= open("speech.mp3", "rb")
4 transcription = client.audio.transcriptions.create(
5     model="gpt-4o-transcribe",
6     file=audio_file
7 )
8 print(transcription.text)
```

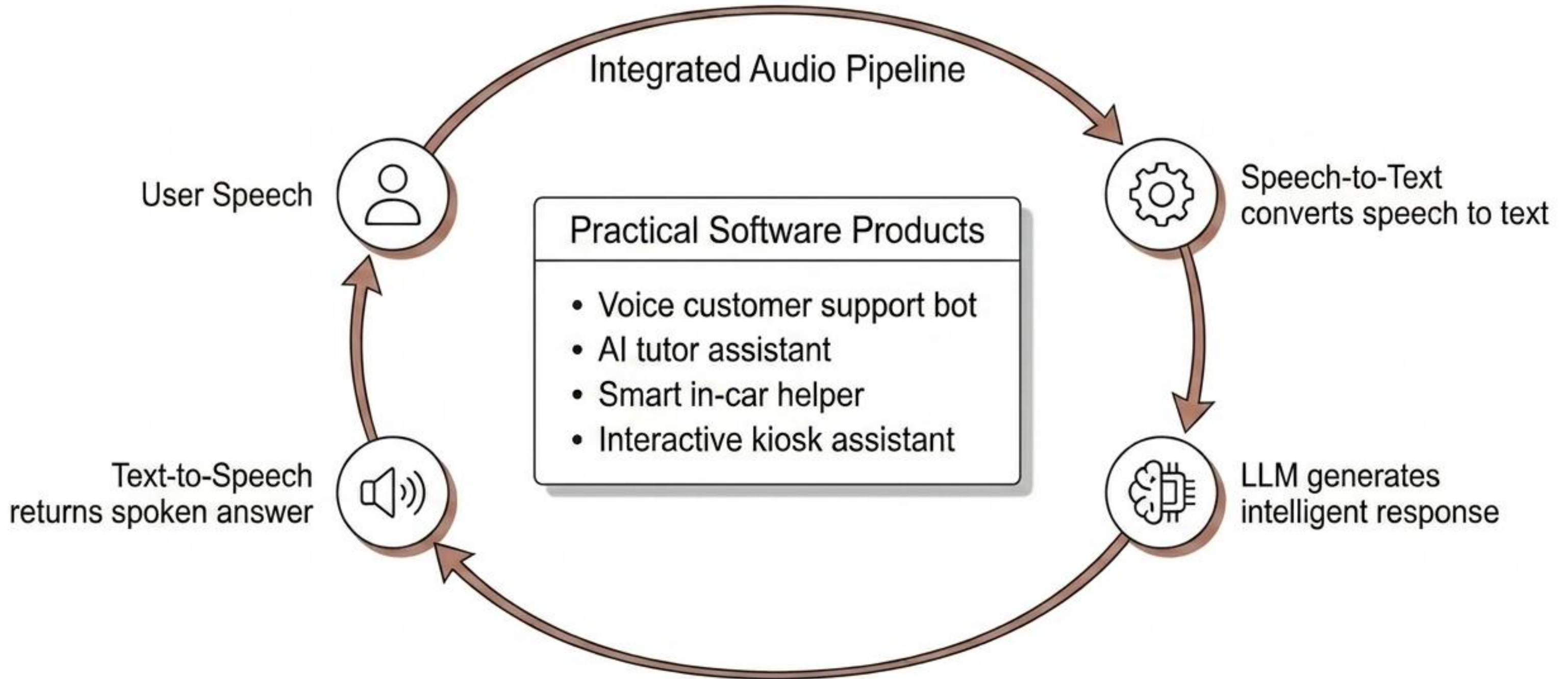
>_

> Execution successful. Output string printed below:

> Hello students, welcome to our amazing course.

Building Voice-Enabled LLM Applications

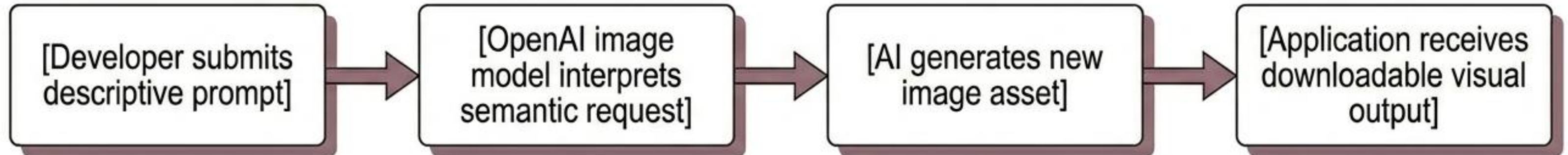
By combining Speech-to-Text, LLM reasoning, and Text-to-Speech, developers can create complete conversational voice systems.



OpenAI Image Generation API

The Image Generation API allows applications to create entirely new visual content directly from textual prompts.

Core Processing Flow



Example Prompt Types

- > futuristic classroom interface
- > modern company logo
- > educational robot mascot
- > mobile app onboarding illustration

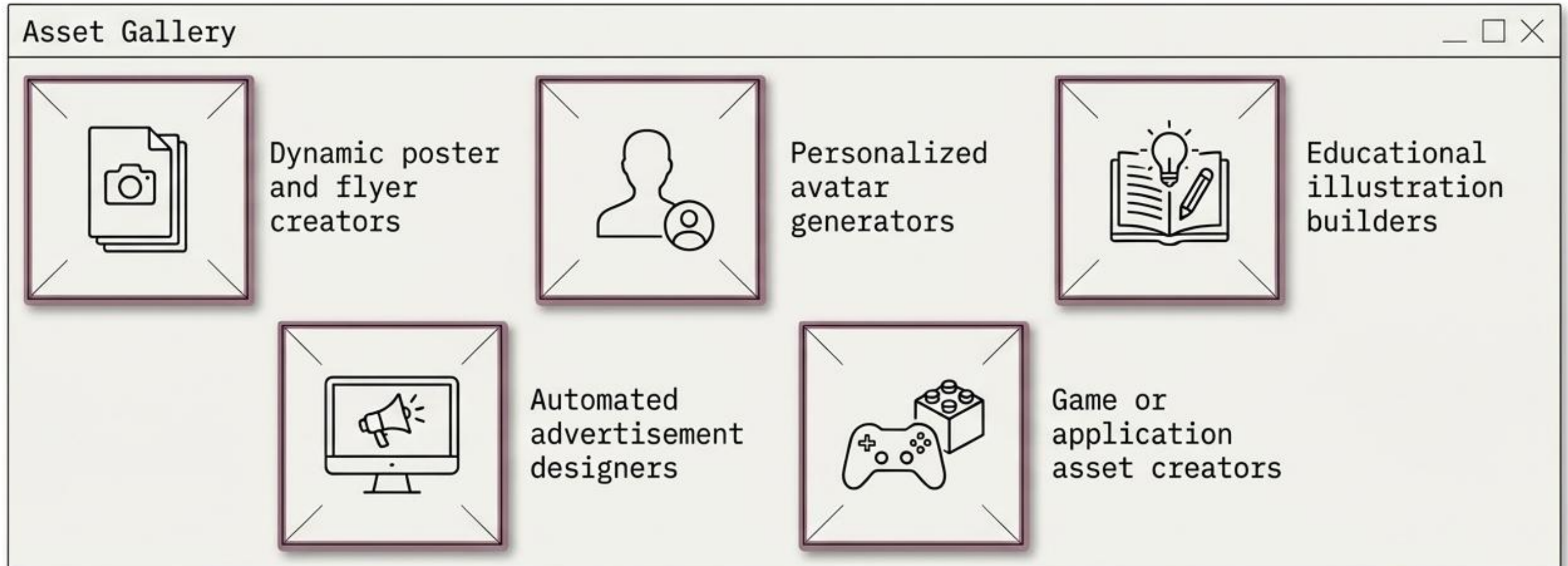
Developer Value:

Applications can automatically generate visual content without manual design work.

Developer Value:

Practical Uses of AI Image Generation

Developers can embed image generation into real software products such as:



The application does not only analyze user input, it can also generate brand-new media content as part of the workflow.

Image Generation Python Implementation

image_gen.py

```
from openai import OpenAI
import base64

client = OpenAI()
prompt = """A children's book drawing of a veterinarian using a
stethoscope to listen to the heartbeat of a baby otter."""

result = client.images.generate(
    model="gpt-image-1-mini",
    prompt=prompt,
    size="1024x1024",
    quality="low"
)

image_base64 = result.data[0].b64_json
image_bytes = base64.b64decode(image_base64)

# Save the image to a file
with open("otter.png", "wb") as f:
    f.write(image_bytes)
```



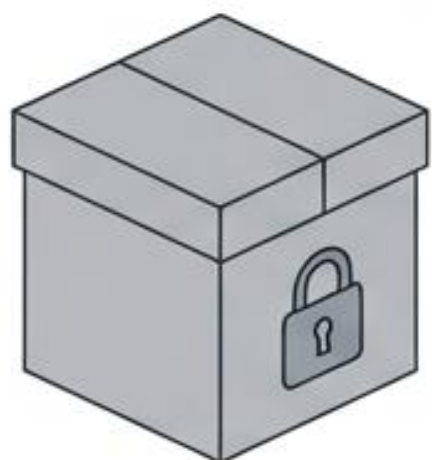
otter.png



Why Multimodal APIs Matter for Modern Developers

Modern AI applications increasingly require more than textual conversation.

Single-Modality Limitation



- Text-only interaction is restrictive
- Users often communicate through images and voice
- Many business workflows involve visual or spoken information

Multimodal Development Advantage

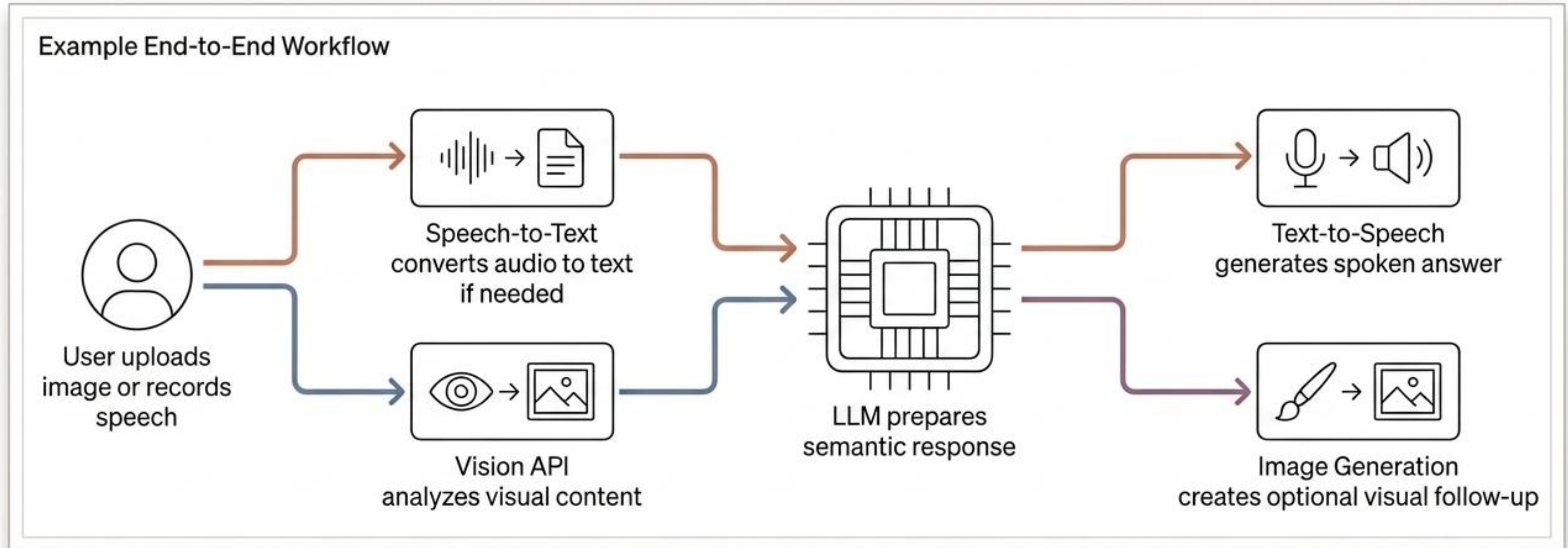


- Richer user interaction
- Broader real-world automation
- More natural application behavior
- Stronger product differentiation

Combining multiple AI modalities transforms isolated API calls into complete intelligent software systems.

Integrated Multimodal Application Architecture

A modern multimodal application combines several OpenAI services into one coordinated processing pipeline.

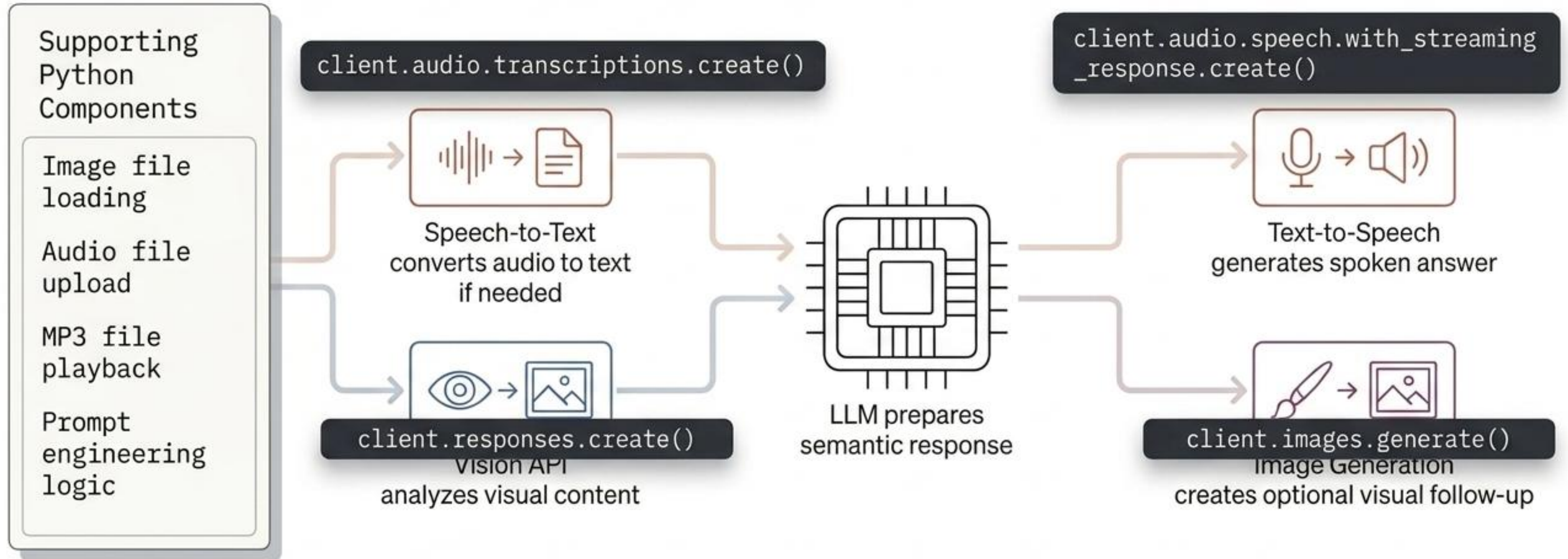


Takeaway

One application can simultaneously see, hear, reason, speak, and create.

Python Implementation Stack for Multimodal Development

All multimodal capabilities in this lesson are accessed through the OpenAI Python SDK.



Takeaway

A single Python backend can orchestrate multiple OpenAI AI services with minimal additional code.

Thank You

